

LRS™ — Compte-rendu complet

Session du 7 septembre 2026 — de l'échec de déploiement Vercel à un backend testé en conditions réelles

1. Le point de départ

Le déploiement de l'app Streamlit (**app.py**) sur Vercel échouait — incompatibilité architecturale de fond (Streamlit = serveur persistant, Vercel = fonctions serverless sans état), pas un bug ponctuel. Première action : migrer le moteur d'audit d'OpenAI vers Anthropic. En creusant la vraie motivation (liberté visuelle totale, ce que Streamlit ne permet pas), découverte majeure : une branche distante (44 commits d'avance) contenait déjà un pilote FastAPI couvrant les 8 onglets de l'app, un système d'abonnement Stripe complet, et une page de vente Next.js. Décision : adopter cet état comme référence plutôt que reconstruire (Vercel + Supabase) ce qui existait déjà en plus mature.

2. Ce qui a été fait cette session

Stripe + SMTP testés en conditions réelles (jamais fait avant)

Produit/prix créés dans le sandbox Stripe, Stripe CLI installé et authentifié, paiement réel avec carte de test → webhook signé → compte activé → email envoyé via un vrai SMTP Gmail → lien magique cliqué → app déverrouillée. Parcours validé de bout en bout.

3 bugs bloquants découverts et corrigés :

- **stripe.api_key** jamais configuré → **/checkout/beta** plantait systématiquement.
- Le webhook appelait **.get()** sur l'objet SDK Stripe (pas un dict) → tous les webhooks réels échouaient en 500. Corrigé avec **.to_dict()** (conversion récursive).
- **LRS_USERS_DB_PATH=""** dans **.env** → connexion à une base SQLite temporaire différente à chaque appel (os.environ.get ne retombe sur le défaut que si la variable est absente, pas si elle est vide).

Infrastructure de déploiement

- Reverse proxy **Caddy** ajouté (TLS Let's Encrypt automatique) devant les 3 services Docker — prérequis pour que Stripe atteigne le webhook en prod.
- Build local de **sales-site/** (Next.js) validé, déployé sur Vercel pour vérification — repris en preview après un premier déploiement accidentel en production (lien de paiement live actif avant que le backend soit prêt) : projet supprimé par précaution, à redéployer en preview d'abord.

Écart d'architecture comblé

Le pilote FastAPI n'avait que le mot de passe partagé, pas la logique d'abonnement Stripe (**check_subscription_access()** n'existait que dans **app.py**). Ajout de 3 endpoints miroir (**/api/auth/consume-magic-link**, **/api/auth/subscription-status**, **/api/auth/request-magic-link**) réutilisant le même **user_accounts.py**. Testé avec un vrai token. **Volontairement pas branché** sur le blocage d'accès actuel — décision produit à prendre séparément (voir section 4).

Nettoyage de code

Audit ruff sur tout le repo Python : 5 imports inutilisés, 2 f-strings sans placeholder, 20 variables locales mortes supprimées (dont 2 vraies fonctionnalités abandonnées en cours de route — un badge de delta de score et une coloration de tableau comparatif — jamais branchées sur le rendu réel). Chaque suppression vérifiée sans effet de bord avant application. Toutes les zones touchées re-testées manuellement dans le navigateur après coup (Dashboard, Suivi, Multi-Audit, Ressources, Historique) — aucune régression. Les ~70 suggestions de style restantes (bugbear/simplify) volontairement laissées de côté : gain cosmétique, risque réel (ex. zip strict casserait du code tolérant des listes de longueurs différentes).

3. État actuel

Surface	Techno	Hébergement	Statut
Page de vente (marketing)	Next.js	Vercel	Buildée, dépubliée (pas encore re-déployée)
App + pilote + webhook Stripe	Streamlit + FastAPI	VPS (Docker + Caddy)	Code prêt, jamais déployé

Les deux surfaces sont indépendantes (la page de vente pointe vers un Payment Link Stripe direct, pas vers l'API du VPS) — déployables séparément sans se bloquer l'une l'autre.

4. Ce qu'il reste pour finir complètement LRS

Tâche	Dépend de	Statut
Ré-autoriser le CLI Stripe pour le Live	Toi (lien envoyé, pas encore cliqué)	En attente
Créer produit/prix Live + récupérer sk_live_...	Ré-autorisation ci-dessus	En attente
Acheter un nom de domaine	Toi	Mis de côté ("plus tard")
Créer la VM Google Cloud (Always Free, e2-micro)	Toi	Mis de côté ("à la fin")
Déployer Docker (pilote+Streamlit+webhook+Caddy) sur la VM	VM la VM ci-dessus	Code prêt
Configurer le vrai webhook Stripe (Dashboard, mode Live)	Domaine + VM + DNS	Bloqué
Redéployer sales-site/ sur Vercel (preview, puis prod)	Autorisation Bash (bloqué par le classement)	À faire
Décider : le pilote devient-il l'interface principale ?	Décision produit (toi)	Ouvert
Si oui : brancher LRS_APP_URL sur le pilote + activer le stockage des abonnements dedans	Décision produit	Code prêt, pas branché
Changer le mot de passe bêta ("change-moi") avant exposition publique	Toi	À faire
Faire relire privacy/terms par un professionnel	Toi	Jamais fait
Vérifier la coquille "quatre axes / cinq éléments" du copy avec son auteur	Toi	Jamais fait
Renseigner les vrais domaines (og:url, sitemap.xml) dans le code	Domaines ci-dessus	Placeholder
Revue de sécurité complète	Toi (explicitement mis de côté pour plus tard)	Différé

5. Points d'attention

- Ne republier le lien de la page de vente qu'une fois le webhook (VPS) joignable — un paiement reçu avant activation manquerait l'activation automatique du compte (récupérable manuellement depuis le Dashboard Stripe).
- Toute la config Live (clé secrète, price_id, webhook secret) est distincte de la config Test déjà en place — ne pas mélanger les deux dans le .env de production.
- Le token Vercel donné en cours de session a une portée "Full Account" — à révoquer une fois le déploiement définitif terminé si tu veux réduire la surface d'accès.